GEKO: An AI Bridge Coach and Playing Partner

MVP Project Report — MAE 301

Team: Alexander Osborn, Sivan Garteiz, Micah Kawamura, Gannon Evers | Arizona State University

1. Executive Summary

Contract bridge is a four-player partnership card game with one of the steepest learning curves in mainstream card games. Today an estimated 600,000 active players worldwide rely on local club meetings, online robot tables, or expensive in-person coaching to improve. The two largest gaps in the existing market are (1) no commercially available product gives the player real, conversational feedback on the calls they make, and (2) the strongest playing AIs (BBO Advance Gib) ignores the player's own conventions and play as if every partner uses the same system.

GEKO is a Bridge-Coach platform that addresses both gaps. The MVP, contained in this repository, is a working web application that lets a single user sit at a virtual bridge table, receive a random deal, bid against strategy-aware AI, and instantly receive a structured natural-language review of their call. Under the hood the MVP combines three components: a strategy-conditioned neural bidding model, a self-play card-play policy, and a fine-tuned small language model that converts model outputs and game state into a coaching response. The system runs end-to-end on a local Python server with a browser-based UI and serves as the foundation for a subscription-based bridge-improvement product.

2. User and Use Case

Target user persona

Our primary persona is "Improving Intermediate Iris" — a recreational bridge player with one to five years of experience who plays one or two club sessions per week, knows a basic system (most often S.A.Y.C. or 2/1), and is frustrated that her improvement has stalled because she rarely receives high-quality post-mortem feedback on the bids and plays she actually made. Secondary personas include curious beginners who want a low-stakes solo learning environment, and serious tournament players who want a sparring partner that can be configured to their partner's exact convention card.

Usage narrative

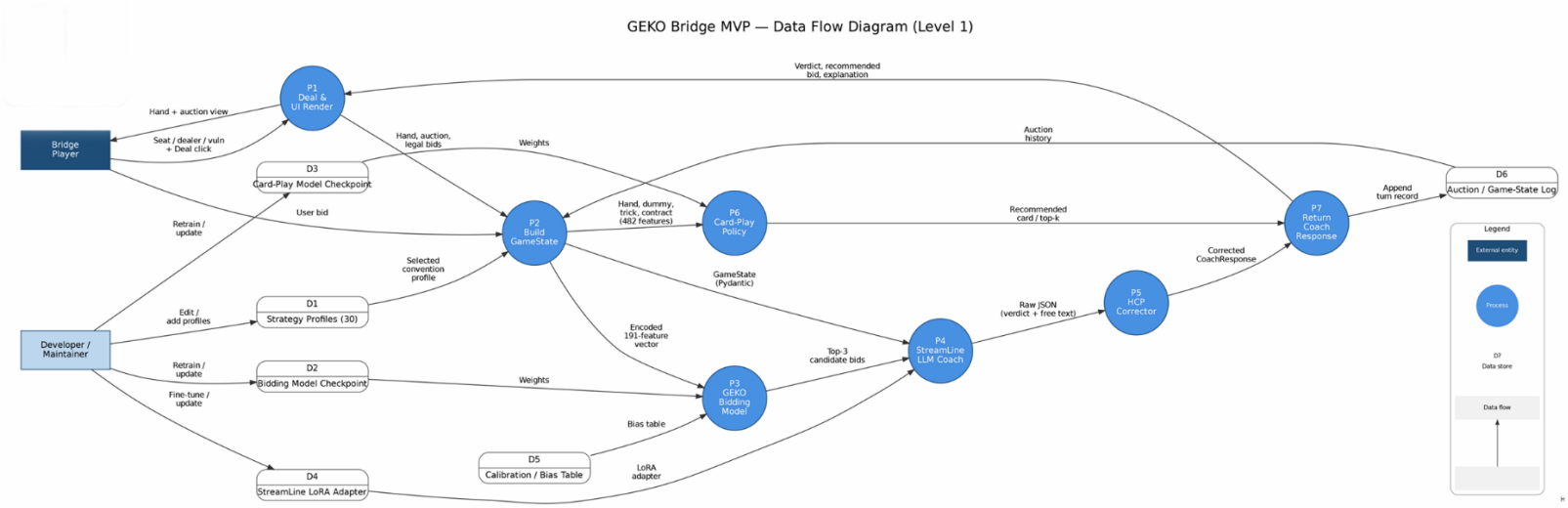
Iris opens her browser to the GEKO Bridge UI. She picks her seat, dealer, and vulnerability and clicks "Deal Random Hand." Her thirteen cards appear in the hand display. The auction begins; when it is her turn, she types a bid (i.e. 1H) and clicks "Play Bid vs GEKO AI." The frontend sends her hand, the auction so far, the legal bids, her chosen bid, and the GEKO model's top three candidate bids to a local /api/coach endpoint. Within seconds she sees a verdict ("reasonable" or "improve"), a recommended

bid, an explanation, the convention-card reasoning behind the recommendation, the risk profile of her own bid, and what her partner is likely to infer. If her bid was already in the model's top three, she gets a confirming response; if not, she sees what the model would have done and why. The auction continues, with the AI seats bidding through the trained model, until the contract is determined.

3. System Design

The MVP is composed of three components that communicate through a thin Python HTTP server. Keeping the components independent lets us swap the playing model or the coaching LLM without touching the UI.

Architecture (data flow)



Component responsibilities

- **Bridge UI (MVP/bridge_ui/):** Single-page HTML/CSS/JS app served by a 90-line Python ThreadingHTTPServer. Handles seat selection, deal randomization, bid entry, and rendering of the LLM's structured review. Static files live alongside server.py so a single python server.py command launches the entire stack.
- **GEKO_PLAYABLE_MODEL:** Self-contained inference package wrapping the strongest tested bidding and card-play checkpoints. Exposes a CLI (playable_bridge_ai.py) for the bid, card, and ai-only modes, plus importable Python classes used by the server.
- **StreamLine:** Standalone inference deployment of the fine-tuned bridge-bid coach. Loads a LoRA adapter on top of SmolLM2-360M-Instruct, builds a chat-template prompt from the GameState, generates a JSON CoachResponse, and runs deterministic post-processing (most importantly HCP correction).

4. Data

Sources

- **SAYC bidding examples:** 1,900 hand-curated bridge auctions in the Standard American Yellow Card system, used as the supervised fine-tuning corpus for the StreamLine LLM coach.
- **Self-play card-play data:** Generated internally by running the policy against itself for roughly three hours on a single machine, scored by tricks-won-by-actor-side. Used to train the advantage-weighted card-play MLP (mvp_card_self_play_policy_3hr_01_model).
- **Bidding outcome data:** Hands played out by the card-play model are scored by their final result; the (auction context, optimal bid) pairs become the training signal for the bidding model (mvp_bid_cardplay_outcome_all_strategies_model). The held-out evaluation set contains 9,919 hands.
- **Strategy profiles:** 30 bundled convention cards expressed as answers to a constrained 75-question form, then converted to numeric vectors and fed to the bidding model as the strategy_declaration feature group.

Cleaning, splits, and visibility policy

Cleaning happens at every boundary: card and bid strings are normalized by normalize_card and normalize_bid to canonical form, the 30 convention cards are pre-converted into the numeric strategy_profiles_numeric.json vector consumed by the bidding model, and incoming LLM payloads are validated against the Pydantic GameState schema before any tokens are generated. Splits are inference-time: the bidding policy uses a held-out 9,919-hand calibration split to fit the per-label biases in calibration.json, the LLM coach was trained for three epochs on a 1,900-example SAYC corpus with a held-out eval set (0.0494 final loss, ~98% token accuracy), and bid_ranker.user_bid_in_top_n short-circuits to a canned reasonable_bid_response whenever the user's bid is already in the model's top three. Visibility is locked down by prompt_builder.build_messages, which only serializes the acting seat's own hand plus public information (auction, vulnerability, dealer, legal bids, top-three candidates, convention card) so neither model can leak hidden information, and coach._correct_hcp then recomputes High Card Points from that hand and silently overwrites any wrong N HCP mention in the free-text fields before the response leaves the server.

5. Models

The MVP pipeline is intentionally split between a small task-specific neural bidding/card-play stack and a fine-tuned small language model that handles natural-language coaching.

Card-play policy

A feed-forward MLP (model_type: card_outcome_policy_mlp) with input dimension 482 and a 512-unit hidden layer. The 482-feature representation encodes the acting hand, visible dummy, current trick, contract, and play history. The training objective is advantage-weighted actor-side tricks with an

advantage temperature of 0.15, weight clip of 5.0, and 0.02 label smoothing — i.e., the model learns to imitate the actions that historically led to higher trick counts for the acting partnership.

Bidding policy

A second MLP (model_type: bid_feature_mlp), input dimension 191, hidden dimension 512, 19 output labels. The feature groups (declared in bid_feature_config.json) include seat one-hot, 52-card presence, HCP, suit lengths and HCP, sorted shape, balanced flag, auction length, trailing passes, last action, last contract, legal-bid summary, dealer and vulnerability context, bidding phase, auction index, and — crucially — a strategy_declaration vector derived from the player's chosen convention profile. This is what allows GEKO to bid as a partner who plays your system rather than a generic system.

StreamLine bid-coach LLM

Base: HuggingFaceTB/SmolLM2-360M-Instruct. Fine-tuning: LoRA adapter (r = 16, alpha = 32) over seven attention/MLP modules, three epochs, on the 1,900-example SAYC corpus. Final evaluation loss: 0.0494; token accuracy: approximately 98%. Adapter weights are merged into the base model at load time via PeftModel.merge_and_unload(), so inference does not require PEFT at runtime.

Workflow design strategies

The coach is wrapped in a small agent-style pipeline (StreamLine/src/bridge_bid_coach/coach.py) rather than a single model call. The pipeline (a) builds a chat-template prompt from a Pydantic GameState that includes the candidate top-three bids from the neural bidding model, (b) generates JSON output with up to three retries, (c) validates the output against a CoachResponse schema, (d) runs a deterministic HCP-correction pass that recomputes the High Card Points from the hand string and uses regex to overwrite any wrong N HCP mentions in the four free-text fields, and (e) falls back to a canned reasonable response when the user's bid is already in the model's top three. The combination of frontier-style structured prompting plus a hard-coded numeric corrector lets a 360M-parameter model behave well above its weight class on the specific subtask of bridge coaching.

6. Evaluation

Evaluation was performed at two levels: token-level accuracy on the LLM and policy-level calibration on the bidding model.

Quantitative metrics

Component	Metric	Value
StreamLine LLM coach	Final evaluation loss (3 epochs)	0.0494
StreamLine LLM coach	Token accuracy	~98%

Component	Metric	Value
StreamLine LLM coach	Latency (CPU / GPU)	~30–60 s / ~2 s per query
Bidding policy	Held-out evaluation hands	9,919
Bidding policy	Output label space	19 bid labels
Card-play policy	Self-play training time	~3 hours, 1 machine
Card-play policy	Feature dimensionality	482
Strategy coverage	Bundled convention profiles	30

Qualitative assessment

On the example hand bundled with the repo (StreamLine/examples/game_state.json — South after 1NT–Pass–2H transfer, holding S KQ984 H 72 D A83 C Q94, with the user choosing 3NT), the coach correctly identifies that 2S (completing the transfer) is the recommended call, returns verdict = improve, and produces a paragraph-level explanation grounded in the convention card. The deterministic HCP corrector catches a known failure mode in which the small base model writes the wrong HCP count in free text (the README shows it converting a model-claimed 18 HCP to the true 14 HCP with no caller-visible artifact).

Error analysis

Inspection of the bidding calibration file shows the most informative failure mode: across 9,919 evaluation hands the model predicts Pass 8,851 times and 3NT 1,044 times, with a long tail of zeros for the in-between bids. The true distribution is far more diverse (122× 1C, 161× 1H, 94× 1NT, 545× 3NT, 193× 4S, …). In other words, the bidding policy as currently trained is heavily under-bidding — a recoverable problem, since the per-bid biases stored in calibration.json are designed to be tuned against this exact distribution.

7. Limitations and Risks

Failure modes

- **Pass-bias in the bidding policy:** Calibration data shows the current bidding model defaults to Pass on the majority of hands. Until the per-label biases are tuned this is the largest single quality gap.
- **Free-text fluency at 360M parameters:** SmolLM2-360M is too small for consistently fluent multi-paragraph explanations; the risk_of_user_bid field occasionally echoes the verdict, which the maintainers' own README flags as a training artifact.

- **Numeric hallucination:** The base model sometimes writes the wrong HCP count. We mitigate this with the deterministic corrector in coach.py, but other numeric facts (suit lengths, vulnerability) are not yet corrected.
- **CPU latency:** Roughly 30–60 seconds per coach query without a GPU. Acceptable for a learning tool but not for live tournament use.

Data and bias risks

The supervised corpus is drawn entirely from SAYC examples, which biases the coach toward American conventions and away from the dozens of systems used internationally (Precision, Polish Club, Acol, etc.). The strategy-profile mechanism partially compensates on the bidding-policy side, but the LLM coach itself has not been multi-system trained. The self-play data for card play has the standard self-play risk: the model can become an expert at exploiting its own weaknesses while remaining vulnerable to play styles it never sees.

Privacy and product risks

The MVP runs entirely locally — no hand data, convention card, or chat traffic leaves the user's machine — so MVP-stage privacy exposure is minimal. The proposal flags three forward-looking risks: limited public availability of high-quality bidding/play data, limited initial access to a paying client base, and compounding training (where the bidding model trained on card outputs and vice versa) that could lead to overfitting to the system's own quirks.

8. Next Steps (2–3 months)

Technical

- **Upgrade the coach to Qwen2.5-1.5B:** The repo already includes the SFT config (config_sft.yaml) for moving from SmolLM2-360M to Qwen2.5-1.5B on a GPU. This is the single highest-leverage change and should resolve most free-text quality issues.
- **Re-tune bidding-policy biases:** Use the existing calibration.json infrastructure to push the model off its Pass default and align predicted bid frequencies with the true held-out distribution.
- **Finish card-play UI integration:** The HTML and Python shells for a card-play panel are scaffolded in index.html (cardplay-panel) but not yet wired to the BEST_CARD_PLAY runner; closing this loop turns the MVP into a complete bid-and-play product.
- **Add a deterministic numeric/legality post-processor:** Extend the HCP corrector to also validate suit length, vulnerability, and bid legality so the coach can never recommend an illegal call.
- **Real bridge-data ingestion:** Negotiate access to BBO or club-level hand records (PBN/LIN files) to broaden the bidding corpus beyond the bundled SAYC set.

Product

- **Convention-card uploader:** Replace the 30 bundled profiles with a UI that lets a player import or build their own convention card and have it converted to the model's numeric strategy vector.
- **Session review mode:** Persist a player's hands and bids and produce a daily/weekly written summary identifying recurring mistakes — the natural upsell beyond per-hand coaching.
- **Beta with a partner club:** Recruit 20–50 club players for a paid beta to validate willingness-to-pay against the projected $10M market opportunity called out in the original proposal.